RETAILMART V3 · SQL

Useful Aggregation Patterns -- Median & DISTINCT ON

— Siraj Sir

📱 WhatsApp Announcement

Useful Aggregation Patterns (Median, Latest-Per-Group)

Today covers two patterns that come up constantly in analyst work: (1) PERCENTILE_CONT for median and percentile calculations, and (2) DISTINCT ON for getting the latest/first record per group without subqueries.

Part 1: PERCENTILE_CONT -- Median, P90, P95
Part 2: DISTINCT ON -- Latest Record Per Group
Part 3: Combining Patterns -- Executive KPI Strips

Both patterns replace clunky workarounds with clean, one-line solutions. After today, you will reach for these automatically whenever someone asks 'what is the median?' or 'show me the latest X per Y.'

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: PERCENTILE_CONT	2 Hours	Median with PERCENTILE_CONT(0.5), P90 and P95 percentiles, WITHIN GROUP (ORDER BY) syntax
Part 2: DISTINCT ON		Latest record per group pattern, DISTINCT ON vs ROW_NUMBER approach, Multiple use cases
Part 3: Executive KPI Strip		Combining PERCENTILE_CONT with pivots, Building dashboard-ready output

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Setup & RetailMart
- ✓ DDL & Data Types
- ✓ Constraints & DML
- ✓ Filtering & Sorting
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregation
- ✓ Joins Part 1
- ✓ Subqueries Part 1
- ✓ Subqueries & CTEs
- ✓ Database Concepts
- ✓ Review & Practice
- ✓ Window Funcs Part 1
- ✓ Aggregation & Frames
- ✓ LAG & LEAD
- ✓ Query Plans
- ✓ Indexing

RECAP

What We Covered Yesterday

- Pivoting with CASE WHEN inside aggregates
- FILTER(WHERE) for cleaner conditional aggregation
- Unpivoting with UNION ALL for long-format data

STORY

Why Not Just Use AVG?

Your CEO asks: 'What is our typical order value?' You report AVG = 8,500. But 5% of orders are 100,000+ electronics purchases that drag the average up. The MEDIAN is 3,200 -- a much more honest answer. Medians resist outliers. That is why analysts need PERCENTILE_CONT.

DEFINITION

PERCENTILE_CONT(fraction) WITHIN GROUP (ORDER BY column)

- Calculates the value at a given percentile position
- fraction = 0.5 is the MEDIAN (50th percentile)
- fraction = 0.9 is P90 (90th percentile -- 90% of values are below this)
- fraction = 0.95 is P95 (95th percentile)
- Uses linear interpolation between adjacent values

PERCENTILE_CONT

```
-- Median

 PERCENTILE_CONT(0.5)
   WITHIN GROUP (ORDER BY column)

-- P90

 PERCENTILE_CONT(0.9)
   WITHIN GROUP (ORDER BY column)

-- Grouped Median

 PERCENTILE_CONT(0.5)
   WITHIN GROUP (ORDER BY col)
 -- Add GROUP BY for per-group medians
```

Median vs Average

```
SELECT
  ROUND(AVG(net_total)::NUMERIC, 2)
    AS avg_order_value,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY net_total)::NUMERIC, 2)
    AS median_order_value,
  ROUND(PERCENTILE_CONT(0.9)
    WITHIN GROUP (ORDER BY net_total)::NUMERIC, 2)
    AS p90_order_value,
  ROUND(PERCENTILE_CONT(0.95)
    WITHIN GROUP (ORDER BY net_total)::NUMERIC, 2)
    AS p95_order_value
FROM sales.orders
WHERE order_status = 'Delivered';
```

If avg = 8,500 but median = 3,200, that tells you the distribution is right-skewed -- a few large orders pull the average up.

Median per Region

```
SELECT
  dr.region_name,
  COUNT(*) AS order_count,
  ROUND(AVG(o.net_total)::NUMERIC, 2)
    AS avg_value,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY o.net_total)::NUMERIC, 2)
    AS median_value
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name
ORDER BY median_value DESC;
```

PRO TIP

PRO TIP

PERCENTILE_CONT is an ordered-set aggregate -- it needs WITHIN GROUP (ORDER BY). It cannot use a regular ORDER BY clause. This is different from window functions which use OVER(ORDER BY).

Practice 1: Median and P95 Order Value per Region

EASY

SCENARIO: The executive team wants to understand order value distribution across regions.

TASK: For each region, calculate the median order value and the P95 order value. Include order count and average for comparison. Only include delivered orders.

HINT: GROUP BY region. Use PERCENTILE_CONT(0.5) for median and PERCENTILE_CONT(0.95) for P95.

Answer

✅ ANSWER

```
SELECT
  dr.region_name,
  COUNT(*)                          AS orders,
  ROUND(AVG(o.net_total)::NUMERIC, 2)
                                    AS avg_value,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY o.net_total)::NUMERIC, 2)
                                    AS median_value,
  ROUND(PERCENTILE_CONT(0.95)
    WITHIN GROUP (ORDER BY o.net_total)::NUMERIC, 2)
                                    AS p95_value
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name
ORDER BY median_value DESC;
```

Practice 2: Median Delivery Time per Courier

MEDIUM

SCENARIO: The logistics team wants the median delivery time, not the average, because a few late deliveries skew the average.

TASK: Calculate median delivery days (delivered_date - order_date) per courier. Also show P90 to identify couriers with long-tail delivery problems.

HINT: Join shipments to orders. Use delivered_date - order_date as the metric inside PERCENTILE_CONT.

Answer

✅ ANSWER

```
SELECT
  sh.courier_name,
  COUNT(*)                          AS shipment_count,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY
      sh.delivered_date - o.order_date)::NUMERIC, 1)
                                    AS median_days,
  ROUND(PERCENTILE_CONT(0.9)
    WITHIN GROUP (ORDER BY
      sh.delivered_date - o.order_date)::NUMERIC, 1)
                                    AS p90_days
FROM sales.shipments sh
JOIN sales.orders o ON sh.order_id = o.order_id
WHERE sh.delivered_date IS NOT NULL
GROUP BY sh.courier_name
ORDER BY median_days;
```

STORY

The Most Recent Order Per Customer

A common analyst request: 'Show me each customer's most recent order.' Without DISTINCT ON, you would use ROW_NUMBER + a CTE + a filter. With DISTINCT ON, it is a single query -- three lines.

DEFINITION

DISTINCT ON (columns) ... ORDER BY columns, sort_col

- Returns ONE row per unique combination of the DISTINCT ON columns
- Which row? The FIRST one after applying ORDER BY
- ORDER BY must START with the DISTINCT ON columns
- PostgreSQL-specific (not standard SQL) but extremely useful

DISTINCT ON

```
-- Latest Per Group

SELECT DISTINCT ON (group_col)
  group_col, other_cols
FROM table
ORDER BY group_col, date_col DESC;


-- Earliest Per Group

SELECT DISTINCT ON (group_col)
  group_col, other_cols
FROM table
ORDER BY group_col, date_col ASC;
```

DISTINCT ON Example

```
-- Latest order per customer (one query, no CTE needed):
SELECT DISTINCT ON (cust_id)
  cust_id,
  order_id,
  order_date,
  net_total,
  order_status
FROM sales.orders
ORDER BY cust_id, order_date DESC;

-- For each cust_id, PostgreSQL:
-- 1. Groups rows by cust_id
-- 2. Sorts by order_date DESC within each group
-- 3. Returns only the FIRST row (most recent order)
```

Comparison

```
-- DISTINCT ON (3 lines, PostgreSQL-specific):
SELECT DISTINCT ON (cust_id)
  cust_id, order_id, order_date
FROM sales.orders
ORDER BY cust_id, order_date DESC;

-- ROW_NUMBER (6 lines, standard SQL):
WITH ranked AS (
  SELECT *, ROW_NUMBER()
    OVER (PARTITION BY cust_id
          ORDER BY order_date DESC) AS rn
  FROM sales.orders
)
SELECT cust_id, order_id, order_date
FROM ranked WHERE rn = 1;

-- Same result. DISTINCT ON is shorter.
-- ROW_NUMBER is portable to other databases.
```

PRO TIP

PRO TIP

The ORDER BY in DISTINCT ON serves two purposes: (1) it determines WHICH row is kept (the first per group), and (2) it determines the output order. The DISTINCT ON columns must appear at the start of ORDER BY.

Practice 3: Latest Order Per Customer

MEDIUM

SCENARIO: The retention team needs each customer's most recent order for a churn analysis.

TASK: Use DISTINCT ON to get each customer's latest order. Show customer_id, order_id, order_date, net_total, and status.

HINT: DISTINCT ON (customer_id) … ORDER BY customer_id, order_date DESC.

Answer

✅ ANSWER

```
SELECT DISTINCT ON (cust_id)
  cust_id,
  order_id,
  order_date,
  net_total,
  order_status
FROM sales.orders
ORDER BY cust_id, order_date DESC;
```

Practice 4: Most Recent Price Per Product

MEDIUM

SCENARIO: The pricing team needs the current (most recent) price for each product from order_items.

TASK: Use DISTINCT ON to get the latest unit_price for each product. Join to orders for the date. Show product_id, latest_date, latest_price.

HINT: DISTINCT ON (oi.prod_id) … ORDER BY oi.prod_id, o.order_date DESC.

Answer

✅ ANSWER

```
SELECT DISTINCT ON (oi.prod_id)
  oi.prod_id,
  o.order_date  AS latest_date,
  oi.unit_price AS latest_price
FROM sales.order_items oi
JOIN sales.orders o ON oi.order_id = o.order_id
ORDER BY oi.prod_id, o.order_date DESC;
```

STORY

The Dashboard Header

Every executive dashboard has a header strip: Median Order Value, P95 Order Value, Total Revenue, Average Days to Deliver. Building this strip combines everything from this week: aggregation, PERCENTILE_CONT, FILTER, and pivoting.

Lab: Executive KPI Strip

Build the KPI strip using median and P95 per region:

Executive KPI Strip (1/2)

```
SELECT
  dr.region_name,
  COUNT(*)                          AS total_orders,
  ROUND(SUM(o.net_total)::NUMERIC, 0)
                                    AS total_revenue,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP
    (ORDER BY o.net_total)::NUMERIC, 0)
                                    AS median_order_val,
  ROUND(PERCENTILE_CONT(0.95)
    WITHIN GROUP
    (ORDER BY o.net_total)::NUMERIC, 0)
                                    AS p95_order_val,
  ROUND(AVG(o.net_total)::NUMERIC, 0)
                                    AS avg_order_val,
  COUNT(*)
    FILTER (WHERE o.order_status = 'Cancelled')
                                    AS cancelled_orders,
  ROUND(COUNT(*)
    FILTER (WHERE o.order_status = 'Cancelled')
```

Executive KPI Strip (2/2)

```
    * 100.0 / NULLIF(COUNT(*), 0), 1)
                                  AS cancel_rate_pct
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
GROUP BY dr.region_name
ORDER BY total_revenue DESC;
```

INTERVIEW

Q: How do you calculate the median in PostgreSQL?

A: Use PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY column). This is an ordered-set aggregate that interpolates between adjacent values to find the 50th percentile. It can be used with GROUP BY for per-group medians.

INTERVIEW

Q: How would you get the latest record per group?

A: Two approaches: (1) PostgreSQL-specific: SELECT DISTINCT ON (group_col) ... ORDER BY group_col, date_col DESC. (2) Standard SQL: ROW_NUMBER() OVER (PARTITION BY group_col ORDER BY date_col DESC) in a CTE, then filter WHERE rn = 1. DISTINCT ON is shorter; ROW_NUMBER is portable.

INTERVIEW

Q: When would you use median instead of average?

A: When the data is skewed (outliers). Order values, salaries, delivery times -- all tend to have long right tails where a few extreme values drag the average up. Median is resistant to outliers and better represents the 'typical' value. Report both: average shows the mathematical mean, median shows the typical experience.

HW 1: Product Price Percentiles by Category

EASY

SCENARIO: The pricing team wants to see the price distribution within each product category.

TASK: For each category, show: product count, min price, median price, avg price, P90 price, max price.

Answer

✅ ANSWER

```
SELECT
  dc.category_name,
  COUNT(*)                          AS products,
  MIN(p.price)                      AS min_price,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY p.price)::NUMERIC, 2)
                                    AS median_price,
  ROUND(AVG(p.price)::NUMERIC, 2)  AS avg_price,
  ROUND(PERCENTILE_CONT(0.9)
    WITHIN GROUP (ORDER BY p.price)::NUMERIC, 2)
                                    AS p90_price,
  MAX(p.price)                      AS max_price
FROM products.products p
JOIN core.dim_brand db ON p.brand_id = db.brand_id
JOIN core.dim_category dc ON db.category_id = dc.category_id
GROUP BY dc.category_name
ORDER BY median_price DESC;
```

HW 2: Latest Salary Per Employee

MEDIUM

SCENARIO: HR needs each employee's most recent salary record.

TASK: Use DISTINCT ON to get the latest salary record per employee. Show employee_id, payment_date, monthly_salary. Then join to employees for name and department.

Answer

✅ ANSWER

```
SELECT DISTINCT ON (sh.employee_id)
  sh.employee_id,
  e.first_name || ' ' || e.last_name AS name,
  d.dept_name,
  sh.payment_date,
  sh.amount AS salary
FROM hr.salary_history sh
JOIN stores.employees e
  ON sh.employee_id = e.employee_id
JOIN core.dim_department d
  ON e.dept_id = d.dept_id
ORDER BY sh.employee_id,
         sh.payment_date DESC;
```

HW 3: Median Revenue Per Region Per Quarter

MEDIUM

SCENARIO: Combine PERCENTILE_CONT with a pivot from Pivoting & FILTER.

TASK: Build a table where rows = regions and columns = Q1 median, Q2 median. Use FILTER to separate quarters and PERCENTILE_CONT for the median.

Answer

✅ ANSWER

```
SELECT
  dr.region_name,
  ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP
    (ORDER BY o.net_total)
    FILTER (WHERE EXTRACT(QUARTER
      FROM o.order_date) = 1)::NUMERIC, 2)
    AS q1_median,
  ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP
    (ORDER BY o.net_total)
    FILTER (WHERE EXTRACT(QUARTER
      FROM o.order_date) = 2)::NUMERIC, 2)
    AS q2_median
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
  AND o.order_date >= '2025-01-01'
  AND o.order_date < '2025-07-01'
GROUP BY dr.region_name
ORDER BY dr.region_name;
```

HW 4: Latest Review Per Product with Rating Stats

HARD

SCENARIO: The product team wants each product's latest review alongside overall rating statistics.

TASK: Use DISTINCT ON to get the latest review per product. Then join to a subquery that calculates avg_rating, review_count, and median_rating per product.

Answer (1/2)

✅ ANSWER

```
WITH latest_review AS (
  SELECT DISTINCT ON (product_id)
    product_id,
    rating       AS latest_rating,
    review_date  AS latest_date
  FROM customers.reviews
  ORDER BY product_id, review_date DESC
),
rating_stats AS (
  SELECT
    product_id,
    COUNT(*)                      AS review_count,
    ROUND(AVG(rating)::NUMERIC, 2) AS avg_rating,
    PERCENTILE_CONT(0.5)
      WITHIN GROUP (ORDER BY rating)
                                  AS median_rating
  FROM customers.reviews
  GROUP BY product_id
)
SELECT
  p.product_name,
  lr.latest_rating,
```

Answer (2/2)

✅ ANSWER

```
  lr.latest_date,
  rs.review_count,
  rs.avg_rating,
  rs.median_rating
FROM latest_review lr
JOIN rating_stats rs USING (product_id)
JOIN products.products p USING (product_id)
ORDER BY rs.review_count DESC;
```

HW 5: Full Executive KPI Strip with MoM

CRAZY

SCENARIO: Build a monthly KPI strip showing median order value, P95 order value, and cancellation rate -- each with MoM change using LAG.

TASK: Combine PERCENTILE_CONT, FILTER, and LAG in a single query. Show month, median_order_val, median_mom_change, p95_order_val, cancel_rate, cancel_rate_change.

Answer (1/2)

✅ ANSWER

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date) AS month,
    ROUND(PERCENTILE_CONT(0.5)
      WITHIN GROUP
      (ORDER BY net_total)::NUMERIC, 0)
                                    AS median_val,
    ROUND(PERCENTILE_CONT(0.95)
      WITHIN GROUP
      (ORDER BY net_total)::NUMERIC, 0)
                                    AS p95_val,
    ROUND(COUNT(*)
      FILTER (WHERE order_status = 'Cancelled')
      * 100.0 / NULLIF(COUNT(*), 0), 1)
                                    AS cancel_rate
  FROM sales.orders
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
  month,
  median_val,
  median_val - LAG(median_val) OVER w
```

Answer (2/2)

✅ ANSWER

```
    AS median_change,
  p95_val,
  cancel_rate,
  ROUND(cancel_rate - LAG(cancel_rate) OVER w, 1)
    AS cancel_rate_change
FROM monthly
WINDOW w AS (ORDER BY month)
ORDER BY month;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

PERCENTILE_CONT for median: PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY col) gives the median. Change 0.5 to 0.9 or 0.95 for other percentiles.

Median resists outliers: Use median when data is skewed. Report both average and median for complete picture.

DISTINCT ON for latest-per-group: SELECT DISTINCT ON (group_col) … ORDER BY group_col, date DESC gives one row per group.

DISTINCT ON vs ROW_NUMBER: DISTINCT ON is shorter (PostgreSQL-only). ROW_NUMBER is standard SQL and portable.

Combine patterns freely: PERCENTILE_CONT + FILTER + LAG = powerful executive KPI strips in a single query.

What is Next

Tomorrow we master Date/Time operations: generate_series for date dimensions, gap-filling for sparse time series, and timezone handling -- the foundation of every time-series dashboard.